

Frozen Lake Reinforcement Learning Assignment Report

1. Introduction

Reinforcement Learning is a machine learning paradigm in which an agent learns decision-making behaviour by interacting with an environment. Unlike supervised learning, RL does not rely on labelled examples or data. Rather, learned through rewards and penalties obtained from sequential actions.

The Frozen Lake problem represents a classical benchmark for reinforcement learning research. The challenge requires an agent to navigate safely through a grid containing hazardous states while maximizing cumulative reward.

The project aims to implement reinforcement learning concepts through the development of a custom environment, a tabular Q-Learning agent, training and evaluation procedures, and visualization tools. The implementation occurs from the first principles of Q-learning and evaluate the agent ability to discover an optimal policy.

2. Problem Formulation

The environment consists of an 8×8 grid with:

- Start state (S)
- Frozen states (F)
- Hole states (H)
- Goal state (G)

The environment contains:

- 64 states
- 4 possible actions per state
- Multiple terminal states (holes and goal)

The task can be formulated as a Markov Decision Process (MDP) defined by states, actions, rewards, and transition dynamics.

The action space defined as:

Action ID	Action
0	Left
1	Down
2	Right
3	Up

2.1 Environment Design

The custom environment was developed from the first principles using Python and it consist the following functionality:

- State initialization
- Action execution
- State transitions
- Reward computation
- Episode termination
- Environment rendering

The environment was implemented as deterministic Markov Decision Process (MDP), where the transitions are determined solely by the chosen action.

The reward design used in the process:

Event	Reward
Goal Reached	+1.00
Fall into Hole	-1.00
Normal Movement	-0.01
Boundary Collision	-0.05

The reward is structured intentionally to accelerate learning:

- Positive rewards encourage reaching the goal.
- Large negative rewards discourage dangerous states.
- Small movement penalties encourage shorter paths.
- Boundary penalties discourage invalid actions.

3. Methodology

3.1 Q-Learning Algorithm

The Q-Learning is temeporal-difference learning algorithm, it estimates the optimal action-value function directly through interaction with the environment.

The expected long-term reward obtained by taking action a in state s is represented by Q-value of state-action pair.

The update rule:

$$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$

where:

- α = learning rate
- γ = discount factor

- r = immediate reward
- s' = next state

3.2 Exploration-Exploitation Strategy

Balancing exploration and exploitation is a fundamental problem in RL. Where the exploration allows the agent to discover better actions, while exploitation leverage current knowledge to maximize reward.

An epsilon-greedy policy was adopted:

- Random action with probability ϵ
- Greedy action with probability $1-\epsilon$

Epsilon decayed from 1.0 to 0.02 throughout training. Thus, the exploration rate gradually decreases over the training period.

3.3 Experiment Setup

The agent was trained using the following hyperparameters:

- Episodes: 20,000
- Learning Rate: 0.12
- Discount Factor: 0.99
- Initial Epsilon: 1.00
- Minimum Epsilon: 0.02
- Epsilon Decay: 0.9995
- Maximum Steps per Episode: 200

The training metrics recorded include:

- Episode reward
- Moving average reward
- Success rate
- Episode length
- Epsilon value
- Cumulative reward

4. Results

4.1 Training Performance

Training metrics reveal substantial improvement over time. Initial episodes produced negative rewards because actions were selected randomly. As the Q-table accumulated experience, rewards increased steadily and stabilized.

The reward curve and moving-average reward curve indicate convergence toward a stable policy.

4.2 Success Rate Analysis

The rolling success rate increased throughout training and eventually stabilized near perfect performance (100%). This behavior indicates successful value propagation and policy optimization.

4.3 Epsilon Decay Analysis

The epsilon-decay curve demonstrates the gradual transition from exploration to exploitation. The final epsilon value of 0.02 preserved limited exploration while allowing the agent to exploit learned knowledge.

Figure 1: Decay Policy

4.4 Value Function Analysis

The value heatmap revealed higher state values along safe routes toward the goal and lower values around hazardous regions. This pattern confirms that the agent successfully learned long-term reward expectations.

4.5 Policy Analysis

The final policy directed the agent around holes while maintaining a short path toward the goal. The extracted policy demonstrates coherent strategic behaviour and confirms successful convergence.

The visualizations including the epsilon decay curve and state-value heatmap and many more were generated and are included in the project repository (results)

5. Evaluation Results

The learned policy was evaluated over 100 independent episodes.

Metric	Result
Success Rate	100%
Average Reward	0.87
Successful Runs	100
Failures	0
Average Steps to Goal	14

These results indicate complete task mastery under the deterministic environment assumptions. The average reward of 0.87 suggests efficient navigation with minimal unnecessary movement penalties.

6. Discussion

The results confirm the theoretical properties of Q-Learning. Early exploration enabled broad environmental coverage, while temporal-difference updates propagated reward information backward through the state space.

Reward shaping contributed significantly to learning efficiency. The small movement penalty encouraged shorter paths, while penalties for holes and boundary collisions discouraged undesirable behaviour.

The final policy demonstrates both safety and efficiency. Achieving a 100% success rate suggests that the learned strategy approximates the optimal policy for the environment.

7. Limitations

Despite excellent performance, several limitations remain:

- Q-Learning scales poorly to large state spaces.
- Performance depends heavily on hyperparameter tuning.
- Frozen Lake is a tabular environment and does not require function approximation.
- Future work could investigate:
 - Deep Q-Networks (DQN)
 - Double DQN
 - SARSA
 - Policy Gradient Methods
 - Actor-Critic Architectures

8. Conclusion

This project successfully implemented a complete Q-Learning framework from first principles and demonstrated that model-free reinforcement learning can solve the Frozen Lake problem effectively. The evaluation results achieved perfect success performance and confirmed convergence toward an optimal navigation strategy.

The project satisfies the assignment objectives while illustrating important reinforcement learning concepts, including value-function learning, temporal-difference updates, exploration–exploitation trade-offs, policy extraction, and empirical evaluation.